ADAPTIVE RL TANK

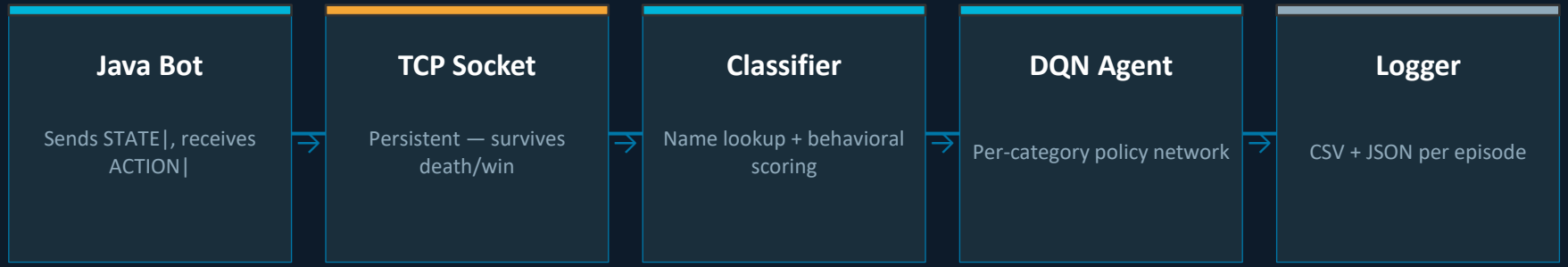
Design & Architecture Breakdown

A multi-policy DQN system that classifies enemy tanks and dispatches specialized learned strategies — built in Python + Java on a persistent TCP architecture.

- Deep Q-Network
- Enemy Classifier
- Per-Category Policies
- Persistent Socket
- Reward Shaping

System Overview

Five components working together in a single persistent session



Data flow per scan:

1 onScannedRobot() fires in Java — builds STATE| pipe-delimited string (11 fields)

2 Python server parses string into a state dict

3 Classifier updates and returns current category

4 Active DQN agent selects action (epsilon-greedy)

5 ACTION| sent back to Java — bot executes movement

6 Reward accumulates in Java until next scan flush

TCP Architecture

One socket. Entire battle.

The Problem

— Robocode creates a new bot instance each round

— onRoundEnded was closing the socket

— Python called conn.close() on done=1

— Connection dropped = training state lost

The Fix

✓ static Socket fields in Java survive class re-instantiation

✓ connectToServer() guards with if (serverConnected) return

✓ onRoundEnded removed — socket never closed mid-battle

✓ Python loops forever on same conn object

✓ Terminal state sends ACTION|0 to keep protocol in sync

```
private static Socket socket;  private static boolean serverConnected = false;  // survives round reset
```

EnemyClassifier — Two Stages

classifier.py — update() called every scan, returns current best category

STAGE 1 — SCAN 1 ONLY

Name Lookup

Strips package prefix from getName() result (e.g. sample.Corners → corners), then runs regex against NAME_TABLE. If confidence ≥ 0.85, category is locked — behavioral stage is skipped.

STAGE 2 — EVERY SCAN (if not name-locked)

Behavioral Heuristics

`avg_vel < 2.0`	→ DEFENSIVE score +2.0
`approach_rate < -10`	→ RUSHER score +1.0
`heading_variance > 0.3`	→ DODGER score +2.0
`dist_variance > 1500`	→ DODGER score +1.0
`avg_vel < 1.0 + far dist`	→ SNIPER score +3.0

DEFENSIVE	Corners, Walls, SpinBot, SittingDuck
RUSHER	RamFire, Fire, Crazy, Target
DODGER	Tracker, Aliens
SNIPER	(no name match — behavioral only)

Why window_size=30?

30 scans ≈ 2–3 seconds of radar sweeps. Long enough to average out noise from single erratic moves. Short enough to reclassify if the enemy changes behavior mid-round. Bearings, distances, velocities and headings are all tracked in separate deques.

StateBuilder — 8-Dimensional Vector

state_vector.py — all values clipped to [-1, 1] before reaching the network

#	Feature	Raw range	Normalization	Output	Why
0	distance	0 – 1000 px	÷ MAX_DISTANCE	0 → 1	Network needs scale parity with energy
1	bearing	-π to π rad	atan2(sin,cos) / π	-1 → 1	atan2 handles angle wraparound correctly
2	enemy energy	0 – 100	÷ MAX_ENERGY	0 → 1	Signals whether to press or retreat
3	my energy	0 – 100	÷ MAX_ENERGY	0 → 1	Survival signal for reward shaping
4	velocity	-8 to 8 px/t	÷ MAX_VELOCITY	-1 → 1	Negative = reversing; sign matters
5	heading	0 to 2π rad	atan2(sin,cos) / π	-1 → 1	Enemy facing direction for prediction
6	my x position	0 – 800 px	÷ BATTLEFIELD_W	0 → 1	Wall proximity awareness
7	my y position	0 – 600 px	÷ BATTLEFIELD_H	0 → 1	Wall proximity awareness

```
np.clip(vec, -1.0, 1.0)  — final safety net catches any out-of-range Robocode values before they reach the network
```

QNetwork — 3-Layer MLP

dqn_agent.py — one instance per category; policy net trains, target net provides stable targets

Input
8
State vec

→

Hidden 1
128
ReLU

→

Hidden 2
128
ReLU

→

Output
4
Q-values

Why 128 × 128 hidden?

Robocode's state space is small (8 features) and structured. A larger network would overfit on sparse per-category data and train slower. Two layers of 128 gives enough capacity to learn non-linear Q-value surfaces without over-parameterizing.

Training Methodology

Three mechanisms that make DQN stable — replay buffer, target network, epsilon decay

10,000	64	200	0.99	0.001
Buffer capacity	Batch size	Target sync steps	Gamma (γ)	Adam LR

Replay Buffer

Stores (s, a, r, s', done) tuples in a deque(maxlen=10000) per category. train() samples 64 random transitions — breaking temporal correlation that would otherwise cause the network to overfit to recent experience and forget earlier lessons.

```
self.buffer.push(state, action, reward,
next_state, done)
```

Target Network

A frozen copy of the policy net. Q-targets are computed using this network — it only syncs every 200 training steps. This prevents the unstable feedback loop where the network chases a moving target, which is the most common cause of DQN divergence.

```
if self._train_steps % TARGET_SYNC == 0:
self.target_net.load_state_dict(...)
```

Epsilon-Greedy Decay

Starts at 1.0 (pure random), decays ×0.995 per episode, floors at 0.05. Early episodes explore freely — the network has no knowledge yet so random play costs nothing. The 0.05 floor keeps a small ongoing exploration budget to prevent permanent local optima.

```
self.epsilon = max(EPSILON_END, self.epsilon
* EPSILON_DECAY)
```

Reward Shaping

All reward signals accumulate in Java between scans and flush atomically with each STATE| message

+3	**onBulletHit**	`reward += 3`	Primary combat objective. +3 outweighs -2 hit-by, so the policy learns that landing shots is worth taking some damage.
-2	**onHitByBullet**	`reward -= 2; moveDirection *= -1`	Penalizes standing still. The direction reversal is a hard-coded evasive behavior layered under the RL policy.
-1	**onHitWall**	`reward -= 1`	Discourages corner-trapping. Small enough not to dominate the combat signal, large enough to matter over many episodes.
+5	**onWin**	`reward += 5; done = true`	Terminal bonus. Sent as the final STATE| with done=1 — server stores this as the terminal transition with a zero next_state.
-5	**onDeath**	`reward -= 5; done = true`	Terminal penalty. Symmetric with win bonus so the value function is centered around survival as a neutral outcome.

Multi-Policy Dispatch

server.py — classifier result indexes into agents dict; only the matching policy trains and acts

DEFENSIVE	RUSHER	SNIPER	DODGER	UNKNOWN

↓ `active_agent = agents[category]`

```
Active DQNAgent → act(state_vec) → remember(...) → train()
```

Isolated experience

A rusher policy's replay buffer never sees sniper encounters. Signal stays clean — the network doesn't have to learn two contradictory strategies simultaneously.

Independent epsilon

Each agent decays its own epsilon. A rarely-seen enemy type stays exploratory longer, while a common one converges faster. Reflects actual training data volume.

Separate weight files

agent.save() writes DEFENSIVE.pt, RUSHER.pt etc. independently. Retraining one policy doesn't touch the others. Weights load at server start via agent.load().

Training Logger & Report

training_logger.py + report.py — persistent across sessions, readable at any time without stopping training

TRAINING LOGGER

```
Per-step: log_step(reward, loss, category,
epsilon, action)
Per-episode: log_episode() → writes CSV row +
updates summary.json
```

✓ episode, timestamp, category, outcome

✓ total_reward, avg_reward, avg_loss

✓ steps, epsilon, duration_s

✓ by_category breakdown in JSON

✓ recent_rewards rolling window (last 20)

REPORT.PY — RUN ANYTIME

```
python report.py
```

→ Overall win rate and total episodes

→ Best episode reward seen

→ Reward trend (up/flat/down) over last 20 eps

→ Per-category: eps, win%, avg reward, avg loss

→ Last 10 episodes detail table

Both files persist across sessions — appending to episodes.csv and rewriting summary.json each episode. Stopping training with Ctrl+C triggers the finally block which calls agent.save() for all five policies before shutdown.

Design Summary

Persistent TCP

Static Java fields + guarded connectToServer() — one socket survives the entire battle including all deaths and wins

Two-Stage Classifier

Name regex fires on scan 1 for instant high-confidence results; behavioral rolling window fills in for unknown bots

Normalized State (8D)

All features scaled to [-1,1] with atan2 angle normalization and a final np.clip() safety net

Per-Category DQN

Five independent 8→128→128→4 networks with separate replay buffers, epsilons, and weight files

Stable Training

Replay buffer (10k), target net (sync/200 steps), gradient clipping (max_norm=1.0), Adam at lr=1e-3

Shaped Rewards

+3 hit, -2 hit-by, -1 wall, ±5 terminal — designed so aggression dominates defense in expectation